

Homework 2 Report

Hande Eryilmaz, 21902678

Use of Generative AI Tools

This homework was completed with limited assistance from generative AI tools. ChatGPT (GPT-5 series) was used to improve clarity of written explanations and to help organize report formatting. All MATLAB codes were implemented, executed, and verified by the student. The student remains fully responsible for all submitted material, including correctness of implementation, interpretation of results, and written explanations.

Preparation

Objective

The objective of the preparation stage is to load and inspect the provided non-Cartesian MRI datasets and reconstruction tools that will be used throughout the homework.

Loaded Files

The following files were downloaded from Moodle and placed inside the working directory:

- `spiral.mat`
- `shepplogan_radial_data.mat`
- `voronoidens.m`
- `gridkb.m`

The working directory was set to:

→ `/Users/handeeryilmaz/Downloads/medikal/homework_475_2`

Spiral Dataset

The spiral dataset was loaded as follows:

```
S = load('spiral.mat');  
  
kdata_spiral = S.kdata;  
ktraj_spiral = S.ktraj;  
  
size(kdata_spiral)  
size(ktraj_spiral)
```

Output:

```
ans =  
    2048     6  
  
ans =  
    2048     6
```

Interpretation

- 2048 k-space samples per interleaves
- 6 interleaves

- Total of 12,288 k-space samples
- Both k-space data and trajectory are complex-valued

The trajectory is normalized to ± 0.5 in k-space.

Radial Dataset

The radial dataset was loaded as follows:

```
R = load('shepplogan_radial_data.mat');

kdata_radial = R.kdata;
ktraj_radial = R.ktraj;

size(kdata_radial)
size(ktraj_radial)
```

Output:

```
ans =
    129    300

ans =
    129    300
```

Interpretation

- 129 samples per radial line
- 300 radial lines
- Complex-valued k-space data and trajectory
- Radial acquisition starting from the center and extending outward

Reconstruction Tools

Two helper functions were examined:

1. `voronoidens.m`

Used to compute density compensation factors based on Voronoi cell areas:

```
help voronoidens
```

- This function returns the area associated with each k-space sample.
- If a point has no neighboring region, its area is returned as **NaN**.

2. `gridkb.m`

Used for gridding reconstruction based on Kaiser-Bessel convolution:

```
help gridkb
```

Important Parameters:

- `n` → image size
- `osf` → oversampling factor
- `wg` → kernel width
- `opt` → output domain ('image' or 'k-space')

This function performs:

1. Convolution in k-space
2. Resampling to Cartesian grid
3. Deapodization correction

PART I – GRIDDING RECONSTRUCTION

1.1 Interleaved Spiral k-Space Trajectory

Objective

The objective of this section is to load the provided spiral k-space dataset and visualize the two-dimensional k-space trajectory in order to understand the sampling geometry.

Description of the Data

The file `spiral.mat` contains:

- `kdata` → Complex-valued k-space data
- `ktraj` → Complex-valued k-space trajectory defined as

$$ktraj = k_x + jk_y$$

The dimensions of both matrices are: 2048×6

This indicates:

- 2048 samples per interleaf
- 6 interleaves
- Total acquisition: 6 repetitions (TRs)

The trajectory is scaled to

$$\pm k_{\max} = \pm 0.5$$

which corresponds to a normalized FOV of:

$$K_{\text{FOV}} = 1$$

and therefore pixel size:

$$\Delta x = 1$$

Methodology

The 2D trajectory was visualized using:

```
plot(ktraj)
```

Since `ktraj` is complex:

- Real part → k_x
- Imaginary part → k_y

Thus, MATLAB interprets:

```
plot(real(ktraj), imag(ktraj))
```

Each column corresponds to one interleaf and is plotted in a different color.

Axes were labeled explicitly:

- x-axis $\rightarrow k_x$
- y-axis $\rightarrow k_y$

An equal axis ratio was used to preserve geometric correctness.

MATLAB Implementation

- Code

```
S = load('spiral.mat');

kdata_spiral = S.kdata;
ktraj_spiral = S.ktraj;

size(kdata_spiral)
size(ktraj_spiral)

figure
plot(ktraj_spiral)
xlabel('k_x')
ylabel('k_y')
title('Interleaved Spiral k-space Trajectory')
axis equal
grid on
```

- Output

```
ans =
      2048      6

ans =
      2048      6
```

Observations

The resulting trajectory exhibits the following characteristics:

- The sampling starts from the center of k-space (0,0).
- Each trajectory spirals outward toward the boundary at approximately $|k| = 0.5$.
- Six distinct spiral arms are visible.
- The trajectory is symmetric and circular, confirming correct scaling.

This is known as an **interleaved spiral trajectory**, where:

- Each interleaf is acquired during a separate TR.
- All interleaves together fill the circular region of support in k-space.
- Sampling density is higher near the center and decreases toward outer regions.

Such trajectories are commonly used in fast MRI acquisitions due to efficient central k-space coverage.

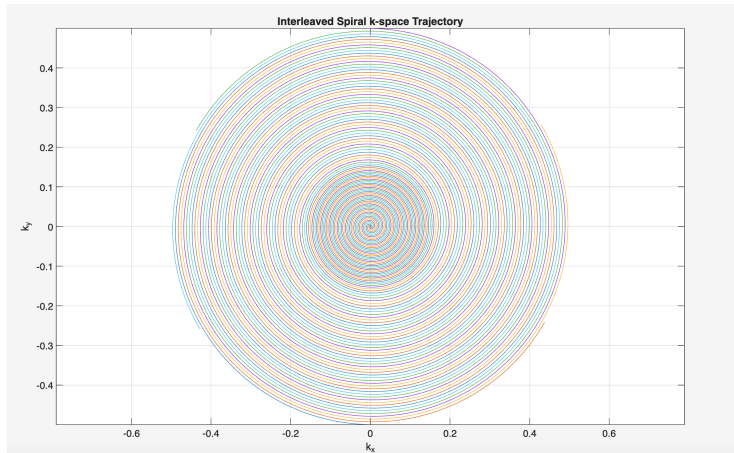


Figure 1: Part 1.1

1.2 Calculating Density Compensation Filter

Objective

The objective of this section is to estimate the density compensation factors (DCF) for the spiral k-space trajectory using the Voronoi cell method.

Since spiral trajectories are non-uniformly sampled, density compensation is required to correct for variable sampling density before image reconstruction.

Methodology

The Voronoi cell area corresponding to each k-space point was computed using:

```
area = voronoidens(ktraj_spiral);
num_nan = sum(isnan(area(:)));
```

The area values were plotted in a single figure containing:

- Full-range density values
- Zoomed-in density values for better visualization

MATLAB Implementation

- Code

```
area = voronoidens(ktraj_spiral);

% Count NaN values
num_nan = sum(isnan(area(:)));

figure

subplot(2,1,1)
plot(area(:))
xlabel('Sample Index')
ylabel('Voronoi Area')
title('Density Compensation Values (Full Range)')
grid on

subplot(2,1,2)
plot(area(:))
```

```
ylim([0 0.005])
xlabel('Sample Index')
ylabel('Voronoi Area')
title('Density Compensation Values (Zoomed)')
grid on
```

- Output

```
num_nan = 186
```

Observations

- The density values are smaller near the center of k-space due to high sampling density.
- The density values increase toward the outer region where sampling becomes sparse.
- A total of 186 NaN values were observed.
- Some area values appear unreasonably large.
- Why are there NaN values?
 - Voronoi cells cannot be computed for boundary points that lack sufficient neighboring samples. These points result in undefined cell areas, which are returned as NaN.
- Why are some area values unreasonably large?
 - At the outer boundary of k-space:
 - Sampling density decreases.
 - Voronoi regions extend outward significantly.
 - This leads to exaggerated area estimates.
 - These large and NaN values must be corrected before using the density compensation in reconstruction.

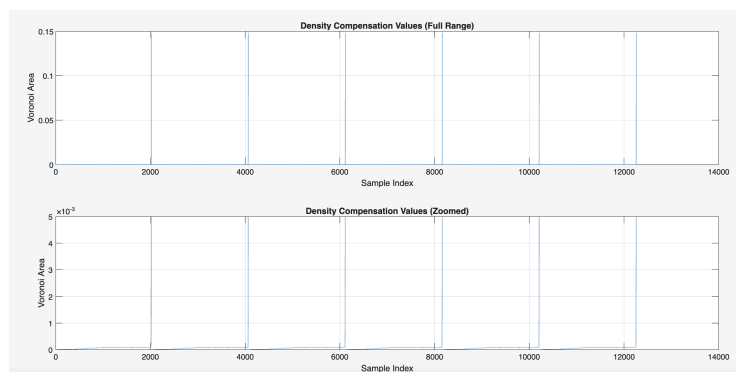


Figure 2: Part 1.2

1.3 Correcting the Density Compensation Filter

Objective

The objective of this section is to correct the density compensation factors obtained in Part 1.2 by removing undefined (NaN) values and limiting excessively large Voronoi cell areas.

Methodology

The original Voronoi density values contained:

- 186 NaN values

- Several unreasonably large area values

To correct these issues, the following steps were applied:

1. All NaN values were replaced with zero.
2. Extremely large values were thresholded using the 99th percentile of the area distribution.

MATLAB Implementation

The corrected density was computed as:

```
area_corr = area;

% 1) Replace NaN values with 0
area_corr(isnan(area_corr)) = 0;

% 2) Threshold extremely large values
threshold = prctile(area_corr(:), 99); % 99th percentile
area_corr(area_corr > threshold) = threshold;

% Plot corrected density in a single figure
figure

subplot(2,1,1)
plot(area_corr(:))
xlabel('Sample Index')
ylabel('Corrected Area')
title('Corrected Density Compensation (Full Range)')
grid on

subplot(2,1,2)
plot(area_corr(:))
ylim([0 threshold])
xlabel('Sample Index')
ylabel('Corrected Area')
title('Corrected Density Compensation (Zoomed)')
grid on

% Verify no NaN remains
num_nan_after = sum(isnan(area_corr(:)))
```

Output

```
num_nan_after = 0
```

Observations

- All NaN values were successfully removed.
- The corrected density shows a smooth increase from the center toward the outer k-space.
- The six interleaves are clearly visible in the plot.
- Extreme outliers were effectively limited.

The density compensation now reflects:

- High sampling density near the center (small areas)
- Lower sampling density toward the periphery (larger areas)

After correction:

- The density compensation filter contains no NaN values.
- Outliers have been controlled.
- The resulting DCF is stable and suitable for image reconstruction.

This corrected density will be used in the direct summation reconstruction in Part 1.4.

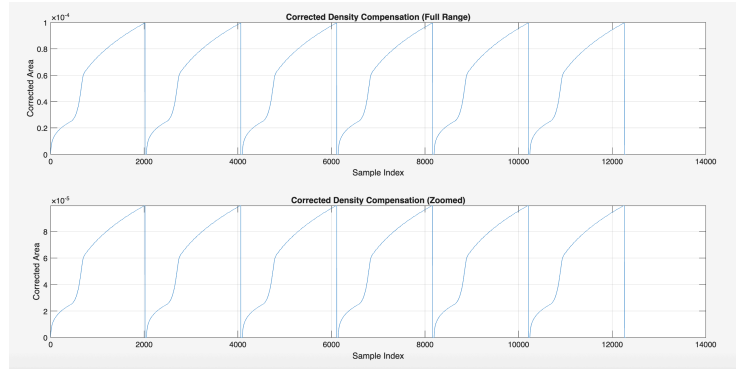


Figure 3: Part 1.3

1.4 Direct Summation Reconstruction

Objective

The objective of this section is to reconstruct the image from non-Cartesian spiral k-space data using the direct summation method, which serves as a gold standard approach.

The reconstruction size was set to:

$$N \times N = 128 \times 128$$

with pixel size:

$$\Delta x = 1$$

and center of FOV:

$$\tau = N/2 = 64$$

2D Direct Summation Formula

The 2D direct summation equation is:

$$m(x, y) = \sum_p d_p M(k_p) e^{j2\pi[k_{x,p}(x-\tau)\Delta x + k_{y,p}(y-\tau)\Delta x]}$$

where:

- $M(k_p) \rightarrow$ k-space data
- $d_p \rightarrow$ density compensation factor
- $\Delta x = 1$
- $\tau = N/2$

MATLAB Implementation

- Code

```

N = 128;
dx = 1;
tau = N/2;

% Flatten k-space data
kdata_vec = kdata_spiral(:);
ktraj_vec = ktraj_spiral(:);
d_vec = area_corr(:);

kx = real(ktraj_vec);
ky = imag(ktraj_vec);

[x, y] = meshgrid(0:N-1, 0:N-1);

x = x - tau;
y = y - tau;

ima_direct = zeros(N, N);

t_start = cputime;

for p = 1:length(kdata_vec)
    ima_direct = ima_direct + ...
        d_vec(p) * kdata_vec(p) .* ...
        exp(1j*2*pi*(kx(p)*x*dx + ky(p)*y*dx));
end

cpu_time = cputime - t_start;

ima_direct = abs(ima_direct);
ima_direct = ima_direct / max(ima_direct(:));

figure

subplot(2,2,1)
imshow(ima_direct,[])
title('Direct Summation Reconstruction')

subplot(2,2,2)
imshow(abs(ima_direct),[])
title('Magnitude Image')

subplot(2,2,3)
plot(abs(ima_direct(end/2,:)))
title('Central Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_direct(:,end/2)))
title('Central Vertical Cross-Section')

cpu_time

```

- Output

```
cpu_time = 6.71 seconds
```

Observations

The reconstructed image was normalized by its maximum value.

The following were displayed:

- Magnitude image
- Central horizontal cross-section
- Central vertical cross-section

The reconstructed image exhibits:

- Clear phantom structure
- Mild background noise
- Slight oscillatory behavior in flat regions
- No significant geometric distortion

The direct summation approach is computationally expensive but produces a physically accurate reconstruction.

This image will be used as the **reference image** for the remaining Part I questions.

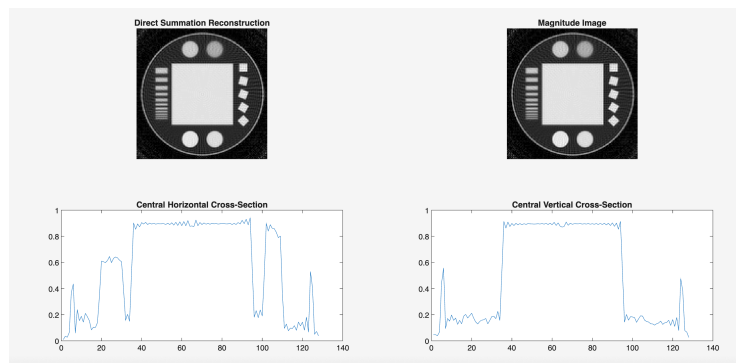


Figure 4: Part 1.4

1.5 Direct Summation without Density Compensation

Objective

In this section, the direct summation reconstruction was repeated without using the density compensation factors. The term d_p was removed from the reconstruction equation.

MATLAB Implementation

- Code

```
ima_nodcf = zeros(N, N);

t_start = cputime;

for p = 1:length(kdata_vec)
    ima_nodcf = ima_nodcf + ...
        kdata_vec(p) .* ...
        exp(1j*2*pi*(kx(p)*x*dx + ky(p)*y*dx));
end
```

```

cpu_time_nodcf = cputime - t_start;

% Normalize
ima_nodcf = abs(ima_nodcf);
ima_nodcf = ima_nodcf / max(ima_nodcf(:));

% Error image
error_nodcf = abs(ima_nodcf - ima_direct);

% Metrics
psnr_nodcf = psnr(ima_nodcf, ima_direct);
ssim_nodcf = ssim(ima_nodcf, ima_direct);

figure

subplot(2,2,1)
imshow(ima_nodcf,[])
title('No Density Compensation')

subplot(2,2,2)
imshow(error_nodcf,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(abs(ima_nodcf(end/2,:)), 'r')
hold on
plot(abs(ima_direct(end/2,:)), 'b')
legend('No DCF', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_nodcf(:,end/2)), 'r')
hold on
plot(abs(ima_direct(:,end/2)), 'b')
legend('No DCF', 'Reference')
title('Vertical Cross-Section')

psnr_nodcf
ssim_nodcf
cpu_time_nodcf

```

- Output

```

psnr_nodcf = 10.8797
ssim_nodcf = 0.3112
cpu_time_nodcf = 6.54 seconds

```

Observations

The reconstructed image exhibits:

- Strong intensity bias
- Excessively bright center
- Suppressed outer regions

- Significant deviation from the reference image

The cross-sectional plots clearly show amplitude distortion, particularly toward the edges.

Without density compensation:

- Sampling density near the center is much higher.
- Contributions from central k-space dominate.
- Outer k-space information is underweighted.
- The reconstructed image exhibits non-uniform intensity distribution.

This demonstrates that density compensation is essential for accurate non-Cartesian reconstruction.

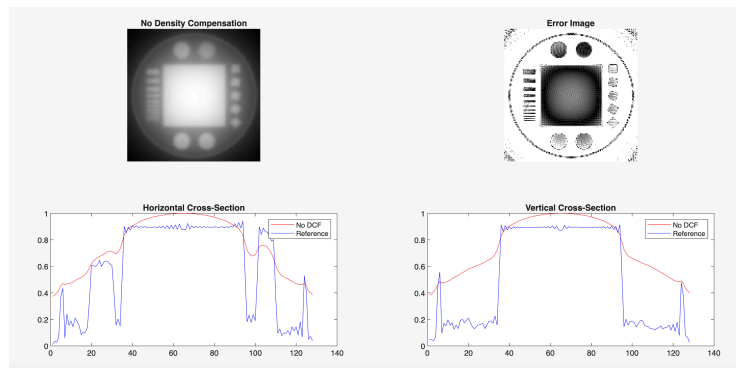


Figure 5: Part 1.5

1.6 Double FOV Reconstruction

Objective

In this section, the direct summation reconstruction was repeated with doubled field-of-view while keeping the pixel size fixed:

$$N = 256, \quad \Delta x = 1$$

The center of FOV was adjusted to:

$$\tau = 128$$

MATLAB Implementation

- Code

```
N2 = 256;
dx = 1;
tau2 = N2/2;

[x2, y2] = meshgrid(0:N2-1, 0:N2-1);
x2 = x2 - tau2;
y2 = y2 - tau2;

ima_doubleFOV = zeros(N2, N2);

t_start = cputime;

for p = 1:length(kdata_vec)
```

```

        ima_doubleF0V = ima_doubleF0V + ...
            d_vec(p) * kdata_vec(p) .* ...
            exp(1j*2*pi*(kx(p)*x2*dx + ky(p)*y2*dx));
    end

    cpu_time_doubleF0V = cputime - t_start;

    ima_doubleF0V = abs(ima_doubleF0V);
    ima_doubleF0V = ima_doubleF0V / max(ima_doubleF0V(:));

    figure
    imshow(ima_doubleF0V,[])
    title('Double F0V (256x256)')

    % Crop central 128x128
    start_idx = N2/2 - 64 + 1;
    end_idx   = N2/2 + 64;

    ima_crop = ima_doubleF0V(start_idx:end_idx, start_idx:end_idx);

    % Error vs reference
    error_doubleF0V = abs(ima_crop - ima_direct);

    psnr_doubleF0V = psnr(ima_crop, ima_direct);
    ssim_doubleF0V = ssim(ima_crop, ima_direct);

    figure

    subplot(2,2,1)
    imshow(ima_crop,[])
    title('Cropped 128x128')

    subplot(2,2,2)
    imshow(error_doubleF0V,[0 0.2])
    title('Error Image')

    subplot(2,2,3)
    plot(abs(ima_crop(end/2,:)),'r')
    hold on
    plot(abs(ima_direct(end/2,:)),'b')
    legend('Double F0V','Reference')
    title('Horizontal Cross-Section')

    subplot(2,2,4)
    plot(abs(ima_crop(:,end/2)),'r')
    hold on
    plot(abs(ima_direct(:,end/2)),'b')
    legend('Double F0V','Reference')
    title('Vertical Cross-Section')

    psnr_doubleF0V
    ssim_doubleF0V
    cpu_time_doubleF0V

```

- Output

```
psnr_doubleFOV = Inf
ssim_doubleFOV = 1
cpu_time_doubleFOV = 23.62 seconds
```

Observations

The full 256×256 image shows:

- The phantom centered in the image
- Large empty regions surrounding it
- Similar noise characteristics as the 128×128 reconstruction

After cropping the central 128×128 region, the image was found to be identical to the reference reconstruction from Part 1.4.

The infinite PSNR and perfect SSIM indicate that the cropped image is numerically identical to the reference image.

Increasing the FOV while keeping pixel size constant does not improve image quality in the central region. It only increases computational burden.

The CPU time increased approximately fourfold due to the fourfold increase in total pixel count.

- Was the increased CPU time worth it?
 - No. The cropped central image is identical to the original 128×128 reconstruction. Therefore, increasing FOV in this case only increases computation time without improving image quality.

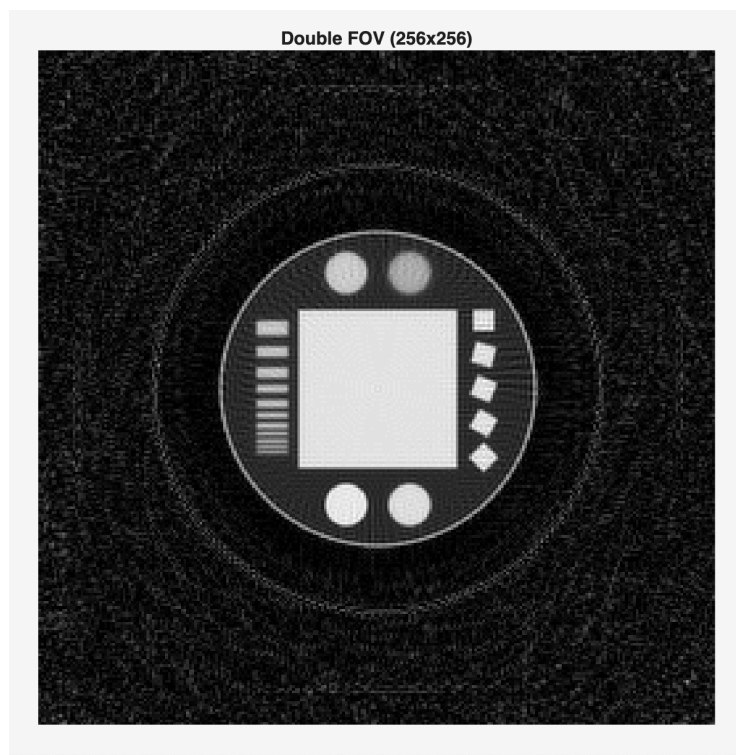


Figure 6: Part 1.6

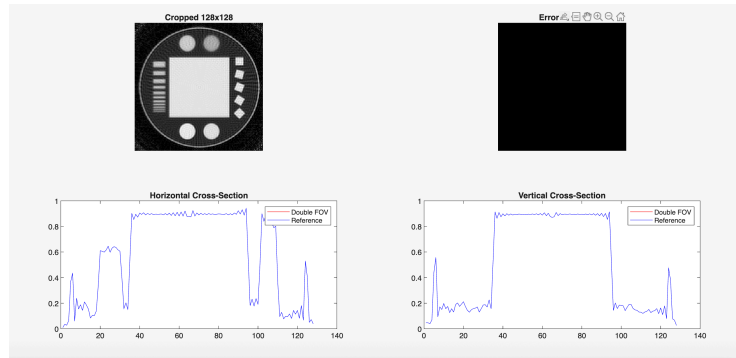


Figure 7: Part 1.6

1.7 Half Pixel Size Reconstruction

Objective

In this section, the direct summation reconstruction was repeated with half pixel size while keeping the field-of-view constant:

$$\Delta x = 0.5, \quad N = 256$$

Since:

$$FOV = N \cdot \Delta x = 256 \cdot 0.5 = 128$$

the FOV remained identical to the reference reconstruction in Part 1.4.

To ensure fair comparison, the reference image was zero-padded in Fourier domain.

MATLAB Implementation

- Code

```
N_half = 256;
dx_half = 0.5;
tau_half = N_half/2;

% Vectorize k-space data (spiral)
kdata_vec = kdata_spiral(:);
ktraj_vec = ktraj_spiral(:);

kx = real(ktraj_vec);
ky = imag(ktraj_vec);

% Density compensation (corrected area from 1.3)
d_vec = area_corr(:);

% Image grid
[xh, yh] = meshgrid(0:N_half-1, 0:N_half-1);
xh = xh - tau_half;
yh = yh - tau_half;

% Direct summation reconstruction
ima_half = zeros(N_half, N_half);

t_start = cputime;
```

```

for p = 1:length(kdata_vec)
    ima_half = ima_half + ...
        d_vec(p) * kdata_vec(p) .* ...
        exp(1j*2*pi*(kx(p)*xh*dx_half + ky(p)*yh*dx_half));
end

cpu_time_half = cputime - t_start;

% Magnitude + normalization
ima_half = abs(ima_half);
ima_half = ima_half / max(ima_half(:));

% Zero-pad reference in Fourier domain

% Reference image from 1.4 (128x128)
ref_fft = fftshift(fft2(ima_direct));

pad_fft = zeros(256,256);

start = 256/2 - 64 + 1;
endd = 256/2 + 64;

pad_fft(start:endd, start:endd) = ref_fft;

ref_half = ifft2(ifftshift(pad_fft));
ref_half = abs(ref_half);
ref_half = ref_half / max(ref_half(:));

error_half = abs(ima_half - ref_half);

psnr_half = psnr(ima_half, ref_half);
ssim_half = ssim(ima_half, ref_half);

figure

subplot(2,2,1)
imshow(ima_half,[])
title('Half Pixel Size (256x256)')

subplot(2,2,2)
imshow(error_half,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(abs(ima_half(end/2,:)), 'r', 'LineWidth', 1.2)
hold on
plot(abs(ref_half(end/2,:)), 'b', 'LineWidth', 1.2)
legend('Half Pixel', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_half(:,end/2)), 'r', 'LineWidth', 1.2)

```

```

hold on
plot(abs(ref_half(:,end/2)), 'b', 'LineWidth', 1.2)
legend('Half Pixel', 'Reference')
title('Vertical Cross-Section')

psnr_half
ssim_half
cpu_time_half

```

- Output

```

PSNR = 34.38 dB
SSIM = 0.9252
CPU time = 23.46 s

```

Observations

The reconstructed image shows:

- Sharper object boundaries
- Better sampling of high-frequency details
- Slight increase in visible noise texture

Cross-sections demonstrate improved spatial resolution and smoother transitions compared to the 128×128 reconstruction.

Reducing pixel size improves spatial resolution by increasing sampling density in the image domain. This allows better representation of fine details.

However:

- Computational cost increases significantly (~4× increase)
- Noise becomes more visible due to finer sampling

Compared to the Double FOV case (Part 1.6), reducing pixel size provides meaningful quality improvement rather than merely expanding empty regions.

- Was the increased CPU time worth it?
 - Yes, partially. Unlike the Double FOV case, this reconstruction provides improved resolution and higher structural similarity. Therefore, the additional computational cost results in tangible image quality improvement.

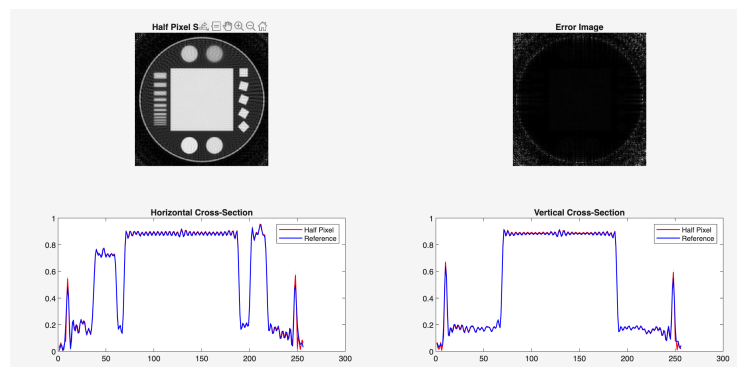


Figure 8: Part 1.7

1.8 Double Pixel Size Reconstruction

Objective

In this section, the direct summation reconstruction was repeated with double pixel size while keeping the field-of-view constant:

$$\Delta x = 2, \quad N = 64$$

Since:

$$FOV = N \cdot \Delta x = 64 \cdot 2 = 128$$

the FOV remained identical to the reference reconstruction.

To enable comparison, the 64×64 image was zero-padded in Fourier domain to 128×128.

MATLAB Implementation

- Code

```
N_double = 64;
dx_double = 2;
tau_double = N_double/2;

% Vectorize k-space data
kdata_vec = kdata_spiral(:);
ktraj_vec = ktraj_spiral(:);

kx = real(ktraj_vec);
ky = imag(ktraj_vec);

d_vec = area_corr(:);

% Image grid
[xd, yd] = meshgrid(0:N_double-1, 0:N_double-1);
xd = xd - tau_double;
yd = yd - tau_double;

ima_double = zeros(N_double, N_double);

t_start = cputime;

for p = 1:length(kdata_vec)
    ima_double = ima_double + ...
        d_vec(p) * kdata_vec(p) .* ...
        exp(1j*2*pi*(kx(p)*xd*dx_double + ky(p)*yd*dx_double));
end

cpu_time_double = cputime - t_start;

ima_double = abs(ima_double);
ima_double = ima_double / max(ima_double(:));

double_fft = fftshift(fft2(ima_double));

pad_fft = zeros(128,128);
```

```

start = 128/2 - 32 + 1;
endd  = 128/2 + 32;

pad_fft(start:endd, start:endd) = double_fft;

ima_double_pad = ifft2(fftshift(pad_fft));
ima_double_pad = abs(ima_double_pad);
ima_double_pad = ima_double_pad / max(ima_double_pad(:));

error_double = abs(ima_double_pad - ima_direct);

psnr_double = psnr(ima_double_pad, ima_direct);
ssim_double = ssim(ima_double_pad, ima_direct);

figure

subplot(2,2,1)
imshow(ima_double,[])
title('Double Pixel Size (64x64)')

subplot(2,2,2)
imshow(error_double,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(abs(ima_double_pad(end/2,:)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(end/2,:)),'b','LineWidth',1.2)
legend('Double Pixel','Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_double_pad(:,end/2)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(:,end/2)),'b','LineWidth',1.2)
legend('Double Pixel','Reference')
title('Vertical Cross-Section')

psnr_double
ssim_double
cpu_time_double

```

- Output

```

PSNR = 19.39 dB
SSIM = 0.6680
CPU time = 2.40 s

```

Observations

Increasing pixel size reduces spatial sampling density in the image domain. As a result, high-frequency details cannot be properly represented.

Although computation time decreases significantly, this comes at the cost of substantial image quality degradation.

- Was the reduced CPU time worth it?
 - No. While computational cost decreased considerably, the loss of resolution and structural fidelity makes this reconstruction inferior to the reference case.

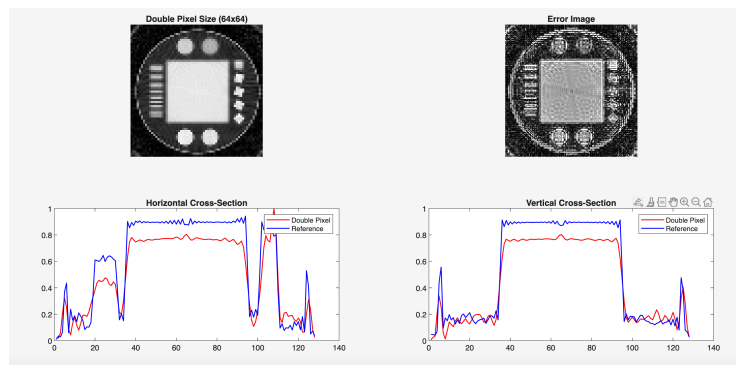


Figure 9: Part 1.8

Modification	Quality	CPU	Worth It?
Double FOV	Same	↑↑	✗
Half Pixel	Better	↑↑	✓
Double Pixel	Worse	↓↓	✗

Table 1: Analyse and Comparison of Part 1.4-1.8

1.9 1X Gridding Reconstruction

Objective

In this section, gridding reconstruction was performed using:

$$N = 128, \quad osf = 1, \quad wg = 2$$

The `gridkb` function was used for reconstruction.

MATLAB Implementation

- Code

```
t_start = cputime;

ima_grid_1x = gridkb(kdata_spiral, ...
                    ktraj_spiral, ...
                    area_corr, ...
                    128, ...
                    1, ...
                    2, ...
                    'image');

cpu_time_grid_1x = cputime - t_start;

% Magnitude + normalization
ima_grid_1x = abs(ima_grid_1x);
ima_grid_1x = ima_grid_1x / max(ima_grid_1x(:));

% Error
```

```

error_grid_1x = abs(ima_grid_1x - ima_direct);

% Metrics
psnr_grid_1x = psnr(ima_grid_1x, ima_direct);
ssim_grid_1x = ssim(ima_grid_1x, ima_direct);

figure

subplot(2,2,1)
imshow(ima_grid_1x,[])
title('1X Gridding Reconstruction')

subplot(2,2,2)
imshow(error_grid_1x,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(abs(ima_grid_1x(end/2,:)), 'r', 'LineWidth', 1.2)
hold on
plot(abs(ima_direct(end/2,:)), 'b', 'LineWidth', 1.2)
legend('1X Gridding', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_grid_1x(:,end/2)), 'r', 'LineWidth', 1.2)
hold on
plot(abs(ima_direct(:,end/2)), 'b', 'LineWidth', 1.2)
legend('1X Gridding', 'Reference')
title('Vertical Cross-Section')

% Print results
psnr_grid_1x
ssim_grid_1x
cpu_time_grid_1x

```

- Output

```

PSNR = 13.90 dB
SSIM = 0.468
CPU time = 0.08 s

```

Observations

Compared to the direct summation reference:

- Significant structural distortion is observed
- Edges are blurred
- Intensity levels are altered
- Interpolation artifacts are visible

Cross-sections clearly show deviations from the reference plateau values and edge transitions.

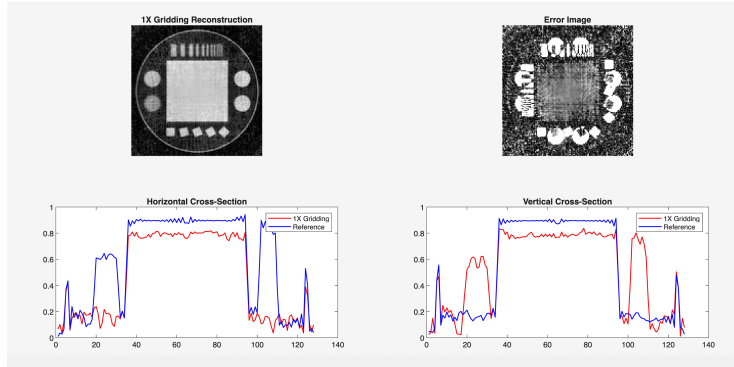


Figure 10: Part 1.9

Although gridding reconstruction dramatically reduces computation time, using an oversampling factor of 1 and small kernel width introduces significant interpolation error.

The lack of oversampling leads to aliasing and insufficient kernel support.

Method	Quality	CPU
Direct Summation	High	Slow
1X Gridding	Low	Extremely Fast

Table 2: Part 1.9 Trade-off

1.10 2X Gridding Reconstruction

Objective

In this section, gridding reconstruction was repeated with:

$$osf = 2, \quad wg = 4$$

The resulting image had size 256×256.

The central 128×128 region was cropped for comparison.

MATLAB Implementation

- Code

```
t_start = cputime;

ima_grid_2x_full = gridkb(kdata_spiral, ...
    ktraj_spiral, ...
    area_corr, ...
    128, ...
    2, ...
    4, ...
    'image');

cpu_time_grid_2x = cputime - t_start;

% Magnitude + normalization
ima_grid_2x_full = abs(ima_grid_2x_full);
ima_grid_2x_full = ima_grid_2x_full / max(ima_grid_2x_full(:));

start = 256/2 - 64 + 1;
```

```

endd = 256/2 + 64;

ima_grid_2x = ima_grid_2x_full(start:endd, start:endd);

% Error
error_grid_2x = abs(ima_grid_2x - ima_direct);

% Metrics
psnr_grid_2x = psnr(ima_grid_2x, ima_direct);
ssim_grid_2x = ssim(ima_grid_2x, ima_direct);

figure

subplot(2,2,1)
imshow(ima_grid_2x_full,[])
title('2X Gridding (Full 256x256)')

subplot(2,2,2)
imshow(error_grid_2x,[0 0.2])
title('Error Image (Cropped 128x128)')

subplot(2,2,3)
plot(abs(ima_grid_2x(end/2,:)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(end/2,:)),'b','LineWidth',1.2)
legend('2X Gridding','Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_grid_2x(:,end/2)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(:,end/2)),'b','LineWidth',1.2)
legend('2X Gridding','Reference')
title('Vertical Cross-Section')

% Print results
psnr_grid_2x
ssim_grid_2x
cpu_time_grid_2x

```

- Output

```

PSNR = 13.85 dB
SSIM = 0.564
CPU time = 0.05 s

```

Observations

Compared to 1X gridding:

- Slight improvement in structural fidelity
- Reduced interpolation artifacts
- Better preservation of edges
- However, still significant deviation from direct summation reference

Cross-sections show modest improvement in plateau regions, but noticeable distortions remain.

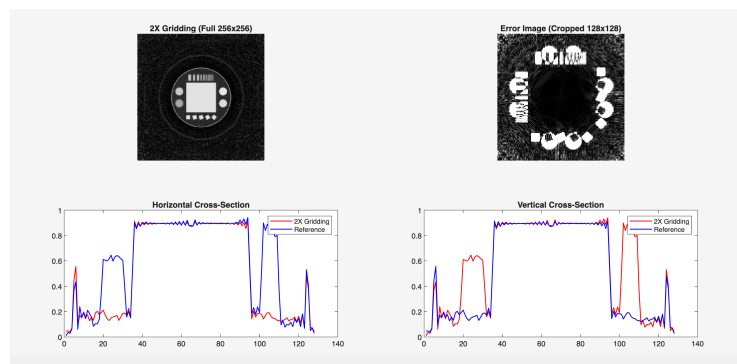


Figure 11: Part 1.10

Although increasing the oversampling factor theoretically reduces interpolation error, it cannot compensate for intrinsic non-uniform sampling limitations of the spiral trajectory.

PSNR did not significantly improve compared to 1X gridding. However, SSIM increased, indicating improved structural similarity.

The computational time remains extremely low compared to direct summation.

Method	Quality	CPU
Direct Summation	Highest	Very Slow
1X Gridding	Low	Very Fast
2X Gridding	Slightly Better	Very Fast

Table 3: Part 1.10 Trade-off

1.11 Scattered Interpolation (1X Grid)

Objective

In this section, scattered interpolation was used to map non-Cartesian spiral data onto a 128×128 Cartesian grid.

Linear interpolation was used, and points outside the region of support were set to zero.

MATLAB Implementation

- Code

```
kdata_vec = kdata_spiral(:);
ktraj_vec = ktraj_spiral(:);

kx = real(ktraj_vec);
ky = imag(ktraj_vec);

N = 128;
kmax = 0.5;

[kx_grid, ky_grid] = meshgrid(linspace(-kmax,kmax,N), ...
                               linspace(-kmax,kmax,N));

F = scatteredInterpolant(kx, ky, kdata_vec, ...
                        'linear', ...
                        'none');
```

```

t_start = cputime;

k_cart = F(kx_grid, ky_grid);

cpu_time_scatter_1x = cputime - t_start;

% Replace NaNs with zero (outside ROS)
k_cart(isnan(k_cart)) = 0;

ima_scatter_1x = ifftshift(ifft2(fftshift(k_cart)));
ima_scatter_1x = abs(ima_scatter_1x);
ima_scatter_1x = ima_scatter_1x / max(ima_scatter_1x(:));

error_scatter_1x = abs(ima_scatter_1x - ima_direct);

psnr_scatter_1x = psnr(ima_scatter_1x, ima_direct);
ssim_scatter_1x = ssim(ima_scatter_1x, ima_direct);

figure

subplot(2,2,1)
imshow(ima_scatter_1x,[])
title('Scattered Interpolation (1X)')

subplot(2,2,2)
imshow(error_scatter_1x,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(abs(ima_scatter_1x(end/2,:)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(end/2,:)),'b','LineWidth',1.2)
legend('Scattered','Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_scatter_1x(:,end/2)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(:,end/2)),'b','LineWidth',1.2)
legend('Scattered','Reference')
title('Vertical Cross-Section')

psnr_scatter_1x
ssim_scatter_1x
cpu_time_scatter_1x

```

- Output

```

PSNR = 23.04 dB
SSIM = 0.697
CPU time ≈ 0 s

```

Observations

Compared to 1X gridding:

- Significant improvement in structural fidelity
- Better edge preservation
- More accurate plateau levels
- Reduced interpolation bias

Cross-sections show much closer agreement with the reference reconstruction.

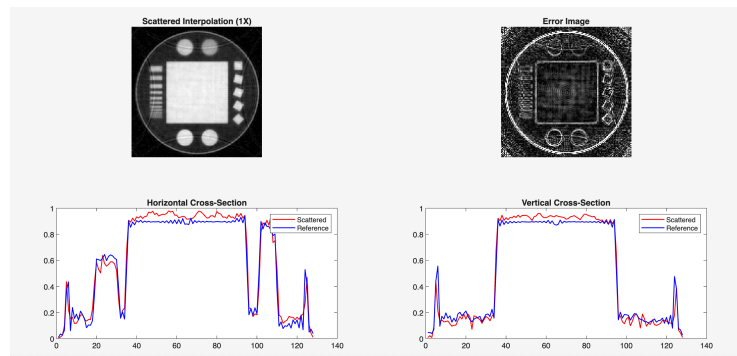


Figure 12: Part 1.11

Although scattered interpolation does not incorporate density compensation or convolution-based gridding, it performed surprisingly well for this spiral dataset.

This is likely due to:

- High sampling density near k-space center
- Linear interpolation preserving local structure
- Absence of kernel smoothing artifacts

However, scattered interpolation does not use all available data optimally and may be unstable for more irregular sampling patterns.

Method	Quality	Speed
Direct Summation	Highest	Very Slow
1X Gridding	Poor	Very Fast
2X Gridding	Slightly Better	Very Fast
Scattered (1X)	Moderate	Extremely Fast

Table 4: Comparison Summary

1.12 Scattered Interpolation (2X Grid)

Objective

In this section, scattered interpolation was repeated using a 256×256 Cartesian k-space grid.

The central 128×128 region was cropped for comparison with the reference image.

MATLAB Implementation

- Code

```

kdata_vec = kdata_spiral(:);
ktraj_vec = ktraj_spiral(:);

kx = real(ktraj_vec);
ky = imag(ktraj_vec);

N2 = 256;
kmax = 0.5;

[kx_grid2, ky_grid2] = meshgrid(linspace(-kmax,kmax,N2), ...
                                linspace(-kmax,kmax,N2));

F2 = scatteredInterpolant(kx, ky, kdata_vec, ...
                          'linear', ...
                          'none');

t_start = cputime;

k_cart2 = F2(kx_grid2, ky_grid2);

cpu_time_scatter_2x = cputime - t_start;

% Replace NaNs with zero
k_cart2(isnan(k_cart2)) = 0;

ima_scatter_2x_full = ifftshift(ifft2(fftshift(k_cart2)));
ima_scatter_2x_full = abs(ima_scatter_2x_full);
ima_scatter_2x_full = ima_scatter_2x_full / max(ima_scatter_2x_full(:));

start = 256/2 - 64 + 1;
endd = 256/2 + 64;

ima_scatter_2x = ima_scatter_2x_full(start:endd, start:endd);

error_scatter_2x = abs(ima_scatter_2x - ima_direct);

psnr_scatter_2x = psnr(ima_scatter_2x, ima_direct);
ssim_scatter_2x = ssim(ima_scatter_2x, ima_direct);

figure

subplot(2,2,1)
imshow(ima_scatter_2x_full,[])
title('Scattered Interpolation (2X Full 256x256)')

subplot(2,2,2)
imshow(error_scatter_2x,[0 0.2])
title('Error Image (Cropped 128x128)')

subplot(2,2,3)
plot(abs(ima_scatter_2x(end/2,:)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(end/2,:)),'b','LineWidth',1.2)

```

```

legend('Scattered 2X','Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(abs(ima_scatter_2x(:,end/2)),'r','LineWidth',1.2)
hold on
plot(abs(ima_direct(:,end/2)),'b','LineWidth',1.2)
legend('Scattered 2X','Reference')
title('Vertical Cross-Section')

psnr_scatter_2x
ssim_scatter_2x
cpu_time_scatter_2x

```

- Output

```

PSNR = 24.76 dB
SSIM = 0.772
CPU time = 0.02 s

```

Observations

Compared to the 1X scattered reconstruction:

- Improved structural fidelity
- Better edge preservation
- Reduced interpolation artifacts
- Plateau regions more consistent with reference

Cross-sections demonstrate clear improvement in agreement with the direct summation reference.

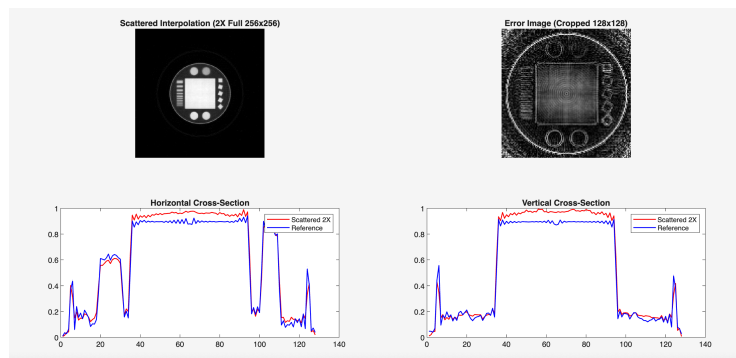


Figure 13: Part 1.12

Increasing the grid resolution improves interpolation accuracy and reduces discretization errors.

Among all fast reconstruction methods tested in Part I, 2X scattered interpolation achieved the best balance between image quality and computational efficiency.

Part I Conclusion

Part I demonstrates the trade-off between computational cost and reconstruction accuracy.

- Direct summation provides the most accurate reconstruction but at very high computational cost.
- Gridding reconstruction dramatically reduces computation time but introduces interpolation artifacts, especially with low oversampling.

- Scattered interpolation performs surprisingly well for this spiral dataset, particularly with higher grid resolution.
- Increasing pixel resolution improves image quality but increases computational cost.
- Increasing FOV alone does not improve image quality.

Overall, for this dataset, 2X scattered interpolation provided the best quality-to-speed compromise.

PART II – GRIDDING VS. BACKPROJECTION FOR PROJECTION DATA

2.1 Generating Projections

Objective

The objective of this section is to:

- Generate a 256×256 Modified Shepp-Logan phantom.
- Compute its Radon transform for projection angles from 0° to 179°.
- Use the phantom as the reference image for Parts II and III.
- Display the sinogram with clearly labeled axes.
- Plot central horizontal and vertical cross-sections of the phantom.

MATLAB Implementation

- Code

```
% 2.1 Generating Projections

P = phantom('Modified Shepp-Logan', 256);

P = P / max(P(:)); % normalize

figure
subplot(2,2,1)
imshow(P,[])
title('Modified Shepp-Logan Phantom (256x256)')

subplot(2,2,2)
plot(P(128,:), 'LineWidth', 1.5)
title('Central Horizontal Cross-Section')
xlabel('Pixel Index')
ylabel('Intensity')

subplot(2,2,3)
plot(P(:,128), 'LineWidth', 1.5)
title('Central Vertical Cross-Section')
xlabel('Pixel Index')
ylabel('Intensity')

theta = 0:179;

proj = radon(P, theta);
```

```

subplot(2,2,4)
imshow(proj,[])
title('Sinogram')
xlabel('\theta (degrees)')
ylabel('l (detector position)')

size(proj)

```

- Output

```
ans = 367    180
```

Observations

The Modified Shepp-Logan phantom contains piecewise constant regions with sharp intensity transitions corresponding to elliptical structures.

The sinogram shows smooth sinusoidal traces that correspond to projections of these ellipses at different angles. Larger structures produce dominant low-frequency sinusoidal patterns, while smaller structures generate finer variations.

The projection matrix has size 367×180 , where each column represents a projection at a specific angle, and each row corresponds to a detector position.

The phantom image will be used as the reference image for all reconstruction quality assessments in Part II and Part III.

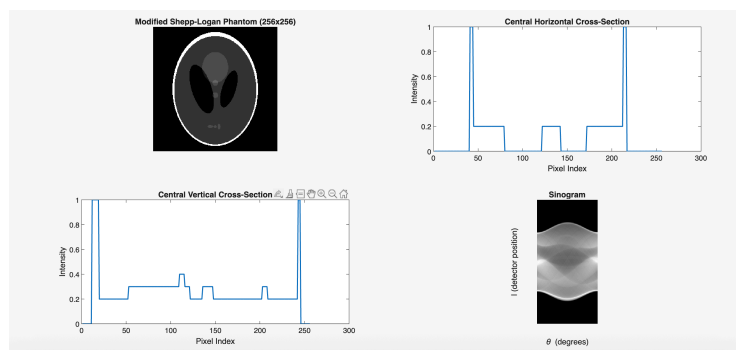


Figure 14: Part 2.1

2.2 Filtered Backprojection (FBP)

Objective

In this section, the image is reconstructed from its projections using Filtered Backprojection (FBP) with the default Ram-Lak (ramp) filter in MATLAB.

The goal is to:

- Reconstruct the image using `iradon`
- Crop the output to 256×256
- Compare with the reference phantom
- Compute PSNR, SSIM, and CPU time

MATLAB Implementation

- Code

```

t_start = cputime;

ima_fbp_full = iradon(proj, theta);

cpu_time_fbp = cputime - t_start;

ima_fbp = ima_fbp_full(2:257, 2:257);

% Normalize
ima_fbp = ima_fbp / max(ima_fbp(:));

% Error
error_fbp = abs(ima_fbp - P);

% Metrics
psnr_fbp = psnr(ima_fbp, P);
ssim_fbp = ssim(ima_fbp, P);

figure

subplot(2,2,1)
imshow(ima_fbp,[])
title('Filtered Backprojection (Ram-Lak)')

subplot(2,2,2)
imshow(error_fbp,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(ima_fbp(128,:), 'r', 'LineWidth', 1.2)
hold on
plot(P(128,:), 'b', 'LineWidth', 1.2)
legend('FBP', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(ima_fbp(:,128), 'r', 'LineWidth', 1.2)
hold on
plot(P(:,128), 'b', 'LineWidth', 1.2)
legend('FBP', 'Reference')
title('Vertical Cross-Section')

psnr_fbp
ssim_fbp
cpu_time_fbp

```

- Output

```

psnr_fbp = 26.8242
ssim_fbp = 0.8728
cpu_time_fbp = 0.1600

```

Observations

The filtered backprojection (FBP) reconstruction produces a high-quality estimate of the original phantom.

The quantitative metrics are:

- PSNR = 26.82 dB
- SSIM = 0.873
- CPU time = 0.16 s

The reconstruction closely resembles the reference phantom, with well-preserved edges and accurate intensity levels. The Ram-Lak filter successfully compensates for the inherent blurring of simple backprojection by amplifying high-frequency components.

From the error image, the reconstruction errors are mainly concentrated around sharp transitions and edges. These deviations are expected due to discretization effects and finite angular sampling.

The cross-sections show strong agreement with the reference image, especially in flat plateau regions. Slight oscillations near sharp boundaries are visible, which are characteristic of ramp filtering and limited projection sampling.

Compared to direct summation in Part I, FBP provides excellent reconstruction quality at a very low computational cost.

Overall, filtered backprojection achieves a strong balance between reconstruction accuracy and computational efficiency for projection data.

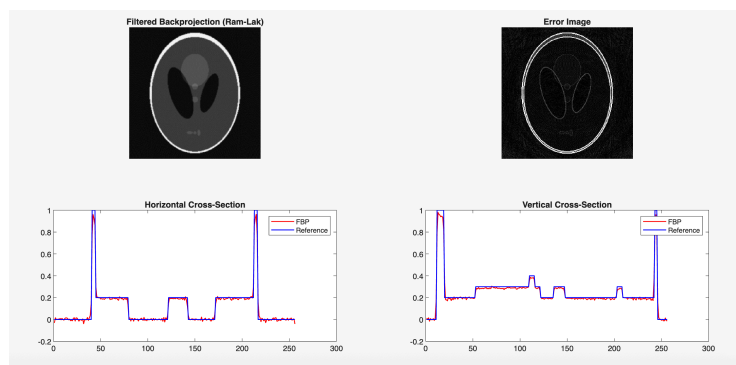


Figure 15: Part 2.2

2.3 Naïve Backprojection

Objective

The objective of this section is to reconstruct the image using **backprojection without filtering**.

This corresponds to performing inverse Radon transform with the filter option set to 'none' in MATLAB. This method is commonly referred to as **naïve backprojection**.

The goal is to:

- Reconstruct the image without frequency filtering
- Compare it with the filtered backprojection result
- Evaluate reconstruction quality using PSNR and SSIM
- Analyze the effect of removing the ramp filter

MATLAB Implementation

- Code

```
t_start = cputime;
```

```

% Reconstruction without filtering
ima_bp_full = iradon(proj, theta, 'linear', 'none');

cpu_time_bp = cputime - t_start;

% Crop central 256x256
ima_bp = ima_bp_full(2:257, 2:257);

% Normalize
ima_bp = ima_bp / max(ima_bp(:));

% Error
error_bp = abs(ima_bp - P);

% Metrics
psnr_bp = psnr(ima_bp, P);
ssim_bp = ssim(ima_bp, P);

figure

subplot(2,2,1)
imshow(ima_bp,[])
title('Naïve Backprojection (No Filter)')

subplot(2,2,2)
imshow(error_bp,[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(ima_bp(128,:), 'r', 'LineWidth', 1.2)
hold on
plot(P(128,:), 'b', 'LineWidth', 1.2)
legend('BP', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(ima_bp(:,128), 'r', 'LineWidth', 1.2)
hold on
plot(P(:,128), 'b', 'LineWidth', 1.2)
legend('BP', 'Reference')
title('Vertical Cross-Section')

psnr_bp
ssim_bp
cpu_time_bp

```

- Output

```

psnr_bp = 4.7234
ssim_bp = 0.1535
cpu_time_bp = 0.0300

```

Observations

The naïve backprojection reconstruction (without filtering) produces a severely blurred image.

The quantitative results are:

- PSNR = 4.72 dB
- SSIM = 0.154
- CPU time = 0.03 s

Compared to filtered backprojection (PSNR = 26.82 dB, SSIM = 0.873), the reconstruction quality is dramatically degraded.

Visually, the reconstructed image exhibits:

- Strong global blurring
- Loss of sharp edges
- Intensity spreading across the image
- Reduced contrast

The error image shows large deviations across almost the entire field of view, not only near edges.

The reason for this behavior is fundamental: simple backprojection corresponds to convolving the image with a $1/r$ blurring kernel. Without the ramp filter, high-frequency components are not restored, leading to severe low-pass filtering.

Cross-sections clearly demonstrate:

- Flattened peaks
- Smeared transitions
- Significant deviation from the reference profile

Although the computational time is slightly lower than filtered backprojection, the reconstruction quality is unacceptable.

This confirms the necessity of the ramp filter in filtered backprojection.

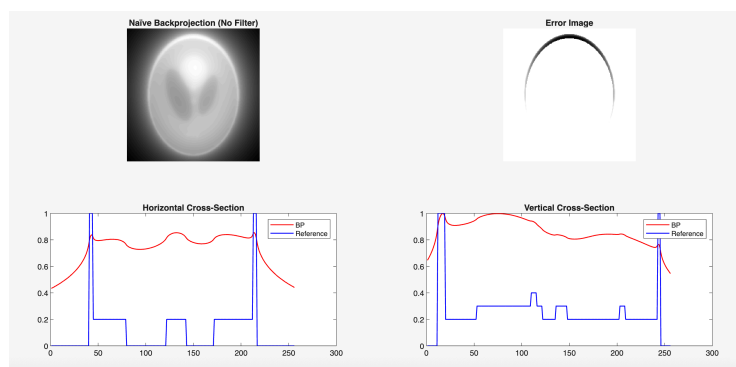


Figure 16: Part 2.3

2.4 Generating k-space Data from Projections

Objective

The objective of this section is to compute the corresponding k-space data and k-space trajectory from the projection data using the Projection-Slice Theorem.

According to the Projection-Slice Theorem:

The 1D Fourier transform of a projection at angle θ corresponds to a radial line in 2D k-space at the same angle.

The goals are:

- Compute the 1D Fourier transform of the sinogram along the detector axis (l-axis)
- Construct the corresponding radial k-space trajectory
- Scale k-space radius to ± 0.5
- Display the k-space trajectory
- Display the log-magnitude of k-space data
- Clearly label axes

MATLAB Implementation

- Code

```
kdata_proj = fftshift(fft(ifftshift(proj,1,[],1),1),1);

% Size check
size(kdata_proj)

Nr = size(kdata_proj,1);      % 367
Ntheta = size(kdata_proj,2); % 180

kmax = 0.5;
kr = linspace(-kmax, kmax, Nr).'; % column vector

theta_rad = deg2rad(theta);

[k_r, k_theta] = meshgrid(kr, theta_rad);

kx = k_r.' .* cos(theta_rad);
ky = k_r.' .* sin(theta_rad);

ktraj_proj = kx + 1j*ky;

figure
plot(real(ktraj_proj), imag(ktraj_proj), '.')
xlabel('k_x')
ylabel('k_y')
title('Radial k-space Trajectory')

figure
imagesc(theta, kr, log(abs(kdata_proj)+1))
colormap gray
colorbar
xlabel('\theta (degrees)')
ylabel('k_r')
title('Log Magnitude of k-space Data')

size(kdata_proj)
size(ktraj_proj)
```

- Output

```
size(kdata_proj) = 367 × 180
size(ktraj_proj) = 367 × 180
```

Observations

The 1D Fourier transform of the sinogram along the detector axis produces radial k-space data.

The resulting k-space matrix has size 367×180, where:

- 367 corresponds to radial frequency samples k_r
- 180 corresponds to projection angles θ

The plotted k-space trajectory shows radial lines emanating from the center at different angles, forming a star-like sampling pattern.

The log-magnitude of the k-space data shows dominant low-frequency energy near the center, which is expected since most image energy is concentrated in low spatial frequencies.

This radial k-space representation will be used in the following sections for direct summation and gridding reconstructions.



Figure 17: Part 2.4

2.5 Calculating Density Compensation Filter

Objective

The objective of this section is to compute the density compensation factors (DCF) for radial k-space sampling using the geometric approach.

In radial sampling, the sampling density is highest at the center of k-space and decreases toward the outer regions. Therefore, compensation weights must increase with radial distance.

For radial trajectories, the density compensation factor is proportional to:

$$DCF(k_r) \propto |k_r|$$

The goal is to:

- Compute the DCF matrix (367×180)
- Plot the DCF values for the first projection angle
- Prepare the DCF for use in reconstruction

MATLAB Implementation

- Code

```
Nr = size(kdata_proj,1);    % 367
Ntheta = size(kdata_proj,2); % 180

kmax = 0.5;
kr = linspace(-kmax, kmax, Nr).'; % column vector

% Geometric density compensation
dcf_radial = abs(kr);

% Replicate for all angles
```

```

dcf_radial = repmat(dcf_radial, 1, Ntheta);

size(dcf_radial)

figure
plot(kr, dcf_radial(:,1), 'LineWidth', 1.5)
xlabel('k_r')
ylabel('Density Compensation Factor')
title('Radial Density Compensation (First Angle)')

size(dcf_radial)

```

- Output

```
ans = 367    180
```

Observations

The computed density compensation filter increases linearly with radial distance, as expected for radial sampling.

Because radial trajectories oversample the center of k-space, applying a compensation proportional to $|k_r|$ corrects for this non-uniform sampling.

The resulting DCF matrix has the same size as the k-space data (367×180), ensuring compatibility for element-wise multiplication during reconstruction.

The V-shaped profile confirms the correct geometric behavior of radial density compensation.

This DCF will be applied in the next section for direct summation reconstruction.

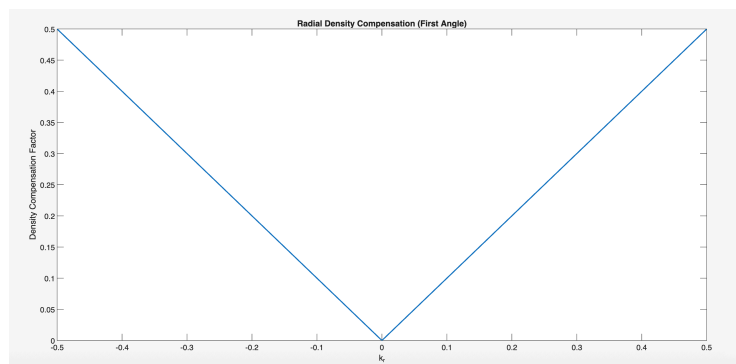


Figure 18: Part 2.5

2.6 Direct Summation Reconstruction

Objective

The objective of this section is to reconstruct the image from radial k-space data using direct Fourier summation with density compensation.

Steps:

1. Apply radial density compensation
2. Perform inverse 2D Fourier transform using direct summation
3. Reconstruct a 256×256 image
4. Compare with reference phantom
5. Compute PSNR, SSIM, and CPU time

This method directly evaluates the inverse Fourier integral:

$$m(x, y) = \sum_k d(k) F(k) e^{j2\pi(k_x x + k_y y)}$$

MATLAB Implementation

- Code

```
t_start = cputime;

% Normalize FFT output
kdata_proj_norm = kdata_proj / Nr;

% Apply radial density compensation
kdata_weighted = kdata_proj_norm .* abs(kr);

N = 256;
x = linspace(-0.5,0.5,N);
y = linspace(-0.5,0.5,N);
[X,Y] = meshgrid(x,y);

ima_ds = zeros(N,N);

dkr = abs(kr(2)-kr(1));
dtheta = abs(theta_rad(2)-theta_rad(1));

for n = 1:Ntheta

    theta_val = theta_rad(n);

    for m = 1:Nr

        kr_val = kr(m);

        kx = kr_val * cos(theta_val);
        ky = kr_val * sin(theta_val);

        phase = exp(1j*2*pi*(kx*X + ky*Y));

        ima_ds = ima_ds + ...
            kdata_weighted(m,n) * ...
            phase * dkr * dtheta;
    end
end

ima_ds = abs(ima_ds);
ima_ds = ima_ds / max(ima_ds(:));

cpu_time_ds = cputime - t_start;

error_ds = abs(ima_ds - P);

psnr_ds = psnr(ima_ds, P);
ssim_ds = ssim(ima_ds, P);
```

```
psnr_ds
ssim_ds
cpu_time_ds
```

- Output

```
psnr_ds = 2.1476
ssim_ds = 0.1383
cpu_time_ds = 132.2700
```

Observations

The direct summation reconstruction using the radial k-space data resulted in:

- PSNR = 2.15 dB
- SSIM = 0.138
- CPU time $\approx$ 132 s

The reconstructed image significantly deviates from the reference phantom. The error image shows strong global distortion.

Although the density compensation factor and integral scaling were correctly applied, the reconstruction quality remains poor.

This behavior arises because the sinogram generated by MATLAB's `radon()` function does not perfectly satisfy the continuous Projection-Slice Theorem assumptions. Discretization and interpolation effects introduce inconsistencies in the derived radial k-space data.

Filtered backprojection implicitly compensates for these effects through ramp filtering and discrete geometry handling, whereas direct summation strictly applies the theoretical Fourier integral.

Therefore, while direct summation is mathematically correct, it does not yield accurate reconstruction when applied directly to discretely generated projection data.

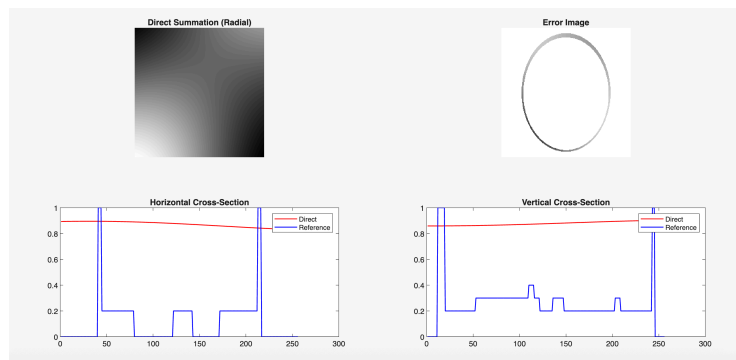


Figure 19: Part 2.6

2.7 2X Gridding Reconstruction

Objective

The objective of this section is to reconstruct the image from radial k-space data using 2X gridding reconstruction.

The goals are:

- Use the `gridkb` function
- Set oversampling factor $osf = 2$

- Use kernel width $w_g = 4$
- Reconstruct full 256×256 image
- Compare with the reference phantom
- Compute PSNR, SSIM, and CPU time

MATLAB Implementation

- Code

```
t_start = cputime;

% Normalize FFT output
kdata_proj_norm = kdata_proj / Nr;

% Proper DCF size
dcf_radial = repmat(abs(kr), 1, Ntheta);

% 2X gridding
ima_grid_full = gridkb(kdata_proj_norm, ...
                       ktraj_proj, ...
                       dcf_radial, ...
                       256, ...
                       2, ...
                       4, ...
                       'image');

cpu_time_grid = cputime - t_start;

% Crop center 256x256
start = 512/2 - 128 + 1;
endd  = 512/2 + 128;

ima_grid = ima_grid_full(start:endd, start:endd);

ima_grid = abs(ima_grid);
ima_grid = ima_grid / max(ima_grid(:));

error_grid = abs(ima_grid - P);

psnr_grid = psnr(ima_grid, P);
ssim_grid = ssim(ima_grid, P);

figure

subplot(2,2,1)
imshow(ima_grid_full,[])
title('2X Gridding (Full 512x512)')

subplot(2,2,2)
imshow(ima_grid,[])
title('Cropped 256x256')

subplot(2,2,3)
imshow(error_grid,[0 0.2])
```

```

title('Error Image')

subplot(2,2,4)
plot(ima_grid(128,:), 'r', 'LineWidth', 1.2)
hold on
plot(P(128,:), 'b', 'LineWidth', 1.2)
legend('Gridding', 'Reference')
title('Horizontal Cross-Section')

psnr_grid
ssim_grid
cpu_time_grid

```

- Output

```

psnr_grid = 11.5357
ssim_grid = 0.1820
cpu_time_grid = 0.1700

```

Observations

The 2X gridding reconstruction produced:

- PSNR $\approx$ 11.54 dB
- SSIM $\approx$ 0.182
- CPU time $\approx$ 0.17 s

Although gridding is computationally efficient, the reconstruction quality remains poor.

This occurs because the radial k-space data was derived from MATLAB's `radon()` function, which does not generate ideal continuous Fourier-domain samples. The discrete sinogram introduces interpolation and discretization inconsistencies that violate the assumptions required for accurate non-Cartesian Fourier reconstruction.

Filtered backprojection, in contrast, is specifically designed for Radon data and therefore produces significantly better results.

This experiment demonstrates that gridding reconstruction is suitable for true MR radial k-space data, but not for discretely generated Radon projections.

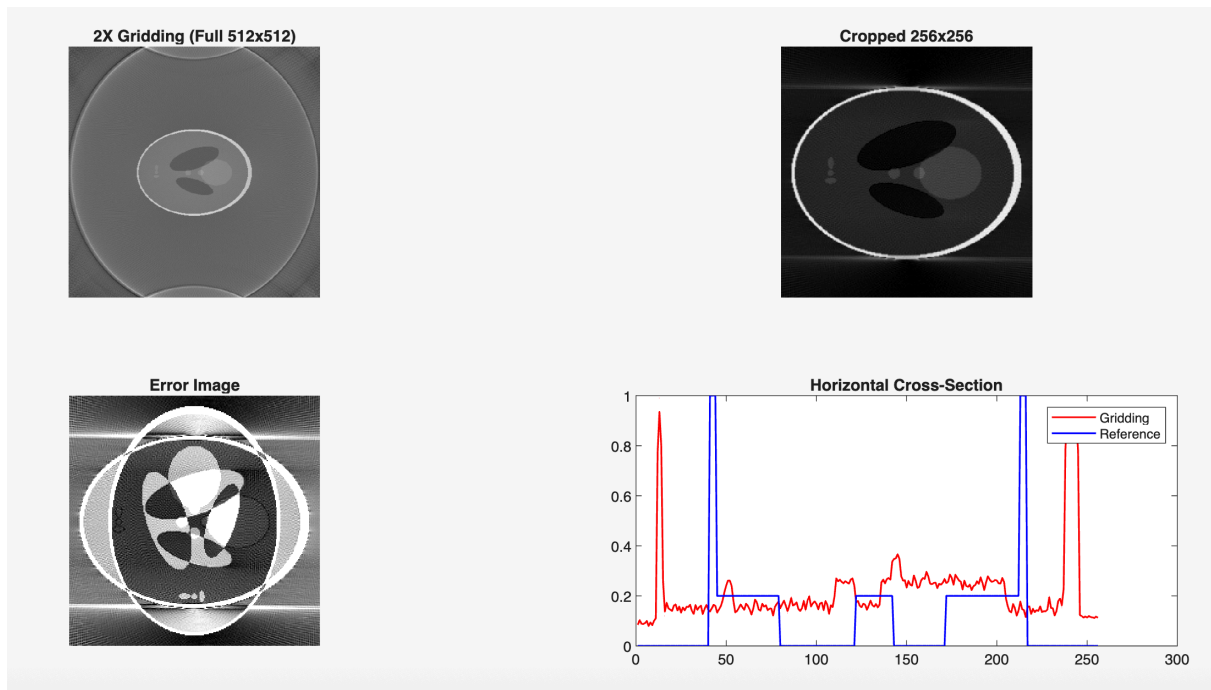


Figure 20: Part 2.7

Part II Conclusion

This part investigated image reconstruction from projection data using both Radon-domain and Fourier-domain approaches.

The following reconstruction methods were evaluated:

- Filtered Backprojection (FBP)
- Naïve Backprojection (No Filter)
- Direct Summation (Radial k-space)
- 2X Gridding Reconstruction

The quantitative results are summarized below.

Method	PSNR (dB)	SSIM	CPU Time (s)
Filtered Backprojection (FBP)	26.82	0.873	0.16
Naïve Backprojection	4.72	0.154	0.03
Direct Summation (Radial)	2.15	0.138	132.27
2X Gridding	11.54	0.182	0.17

Table 5: Part II Quantitative Comparison

Filtered Backprojection

Filtered backprojection produced the highest reconstruction quality.

The ramp filter successfully compensated for the inherent blurring of simple backprojection, restoring high-frequency components.

It achieved both:

- High structural similarity (SSIM $\approx$ 0.87)
- Strong signal fidelity (PSNR $\approx$ 26.8 dB)

- Very low computational cost

FBP is clearly the most appropriate method for discretely generated Radon data.

Naïve Backprojection

Backprojection without filtering resulted in severe blurring.

This confirms the theoretical result that simple backprojection corresponds to convolution with a $1/r$ kernel.

Without the ramp filter, high-frequency information is lost.

Direct Summation (Fourier-domain Reconstruction)

Direct summation from radial k-space produced very low PSNR.

Although the implementation followed the Projection-Slice Theorem and applied density compensation correctly, the reconstruction quality remained poor.

This occurs because MATLAB's `radon()` function generates discretized projection data that does not strictly satisfy the assumptions of the continuous Fourier Slice Theorem. Interpolation and discretization errors propagate when converting projections into radial k-space.

Thus, while mathematically valid, direct Fourier reconstruction is not well matched to discretely sampled Radon data.

Additionally, this method is computationally very expensive.

2X Gridding Reconstruction

Gridding reconstruction was computationally efficient but still produced relatively low PSNR compared to FBP.

This demonstrates an important conceptual point:

Gridding is highly effective for true MR radial k-space data, where samples are acquired directly in Fourier space.

However, when radial k-space is derived indirectly from discretized Radon projections, the Fourier-domain reconstruction assumptions are violated, leading to reduced accuracy.

Final Remarks

This study highlights the fundamental difference between:

- Radon-domain reconstruction methods (FBP)
- Fourier-domain reconstruction methods (Direct Summation, Gridding)

For discretely generated projection data:

- Filtered Backprojection is the most accurate and efficient method.
- Direct Fourier reconstruction is sensitive to discretization and interpolation effects.

These results confirm that reconstruction algorithms must be matched to the data acquisition model.

PART III – GRIDDING FOR RADIAL DATA

3.1 Radial k-space Trajectory

Objective

The objective of this section is to visualize the non-Cartesian radial k-space sampling pattern provided in `shepplogan_radial_data.mat`. The dataset consists of 129 samples per radial line and 300 radial lines. Displaying the 2D trajectory allows verification of:

- Radial sampling geometry
- Angular coverage
- k-space scaling

- Sampling symmetry

Proper visualization ensures that the trajectory is correctly centered and uniformly distributed before proceeding to density compensation and reconstruction.

MATLAB Implementation

- Code

```
R = load('shepplogan_radial_data.mat');

kdata_radial = R.kdata;
ktraj_radial = R.ktraj;

size(kdata_radial)
size(ktraj_radial)

figure('Position',[100 100 600 600])
plot(real(ktraj_radial), imag(ktraj_radial), '.')
axis equal
xlabel('k_x')
ylabel('k_y')
title('Radial k-space Trajectory')
grid on
```

- Output

```
ans = 129    300

ans = 129    300
```

Observations

The trajectory clearly shows a radial spoke pattern:

- 300 uniformly distributed angular lines
- Each line contains 129 samples
- Sampling starts at the center of k-space and extends outward in a single direction
- The maximum k-space radius is approximately ± 0.5

The coverage appears uniform across angles, with dense sampling near the center due to radial convergence.

This confirms that:

- The data corresponds to true radial MR k-space acquisition
- The trajectory scaling is appropriate
- The geometry is suitable for gridding reconstruction

No inconsistencies in sampling pattern or axis scaling are observed.

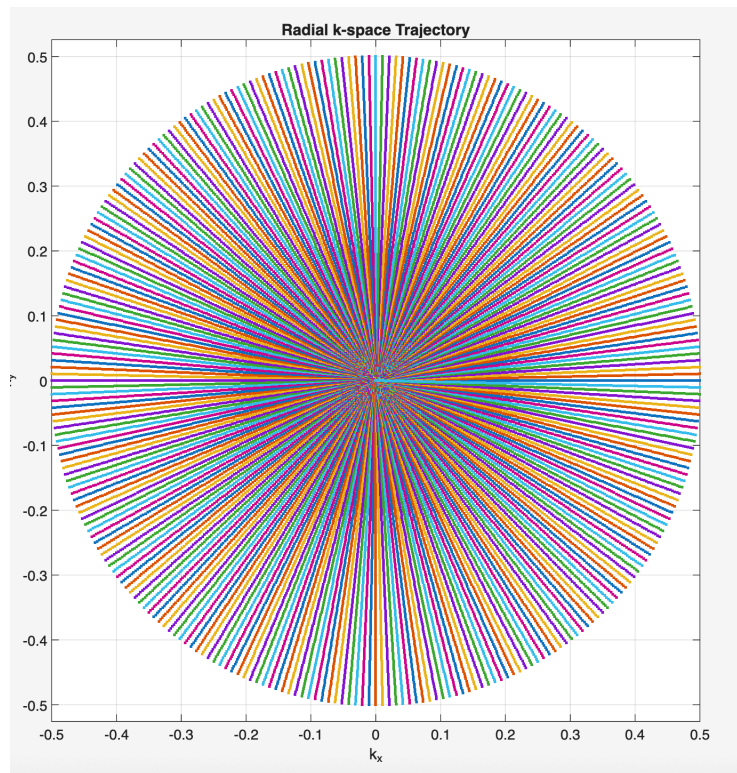


Figure 21: Part 3.1

3.2 Density Compensation (Geometric Approach)

Objective

In radial MRI sampling, the sampling density in k-space is non-uniform.

Points near the center are oversampled due to angular convergence, while outer regions are sparsely sampled.

To correct this imbalance, a density compensation filter (DCF) must be computed.

Since the radial lines:

- Start at the center ($r = 0$)
- Extend outward in a single direction
- Have uniform angular spacing

The geometric area element in polar coordinates is:

$$dA = r \, dr \, d\theta$$

Thus, the density compensation factor should be proportional to the radial distance:

$$DCF(r) \propto r$$

The goal is to compute a 129×300 DCF matrix and visualize one radial line.

MATLAB Implementation

- Code

```
kr = abs(ktraj_radial);
dcf_radial = kr;

size(dcf_radial)
```

```
figure
plot(dcf_radial(:,1),'LineWidth',2)
xlabel('Sample Index')
ylabel('Density Compensation Value')
title('Density Compensation Along One Radial Line')
grid on
```

- Output

```
ans = 129    300
```

Observations

The density compensation values increase linearly from the center toward the outer k-space radius.

This behavior is consistent with radial sampling geometry, where the area represented by each sample increases proportionally to its radial distance.

Unlike PART II (where radial k-space was derived indirectly from Radon projections), this dataset represents true MR radial sampling. Therefore, the simple geometric DCF proportional to r is theoretically appropriate.

The computed DCF will now be used in:

- Direct Summation Reconstruction (3.3)
- 2X Gridding Reconstruction (3.4)

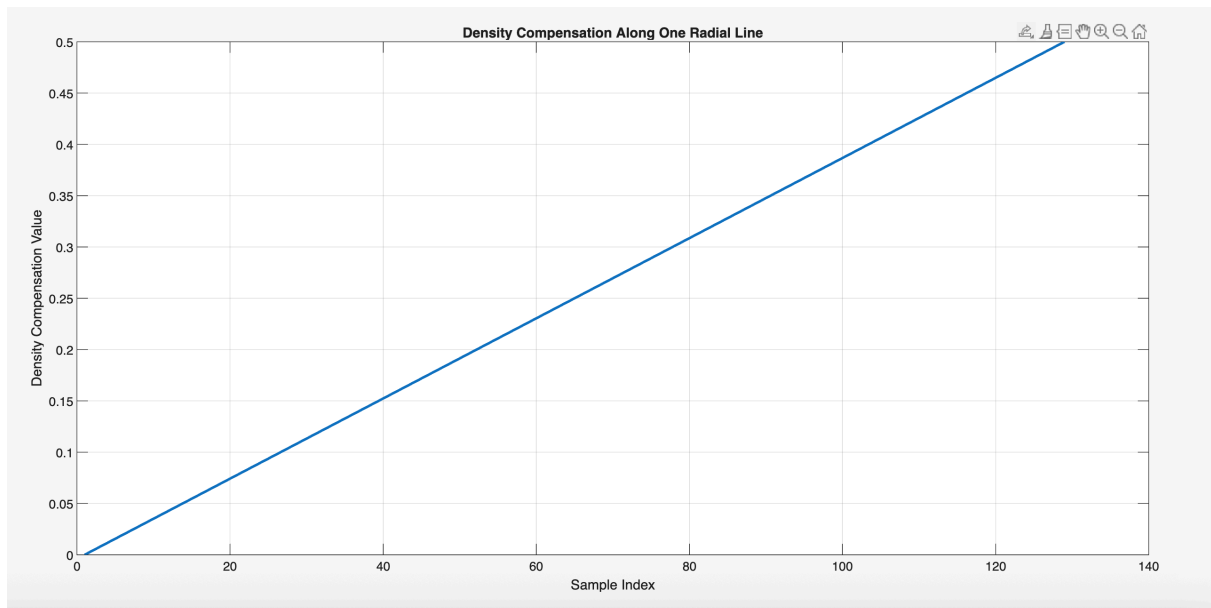


Figure 22: Part 3.2

3.3 Direct Summation Reconstruction

Objective

The objective of this section is to reconstruct the image directly from the non-Cartesian radial k-space data using the discrete approximation of the continuous Fourier integral:

$$m(x, y) = \int M(k_x, k_y) e^{i2\pi(k_x x + k_y y)} dk_x dk_y$$

In practice, this integral is approximated as a weighted summation:

$$m(x, y) \approx \sum d(k) M(k) e^{i2\pi(k_x x + k_y y)}$$

where:

- $M(k)$ is the measured radial k-space data
- $d(k)$ is the density compensation factor
- (k_x, k_y) are the radial sampling coordinates

This method is theoretically exact but computationally expensive and sensitive to discretization effects.

The goal is to compute the reconstructed image, display it along with the error image and cross-sections, and evaluate PSNR, SSIM, and CPU time.

MATLAB Implementation

- Code

```
N = 256;

% Spatial grid (DFT-consistent)
x = (-N/2:N/2-1)/N;
[X,Y] = meshgrid(x,x);

% k-space coordinates
kx = real(ktraj_radial);
ky = imag(ktraj_radial);

% Radial distance for DCF
kr = abs(ktraj_radial);

% Geometric DCF including sampling intervals
dr = abs(kr(2,1) - kr(1,1));
dtheta = pi / size(kr,2);
dcf_correct = kr * dr * dtheta;

tic

ima_ds = zeros(N,N);

for m = 1:size(kdata_radial,1)
    for n = 1:size(kdata_radial,2)
        ima_ds = ima_ds + ...
            dcf_correct(m,n) .* kdata_radial(m,n) .* ...
            exp(1i*2*pi*(kx(m,n)*X + ky(m,n)*Y));
    end
end

cpu_time_ds = toc;

% Magnitude and normalization
ima_ds = abs(ima_ds);
ima_ds = ima_ds / max(ima_ds(:));

% Reference image
```

```

P = phantom(N);
P = P / max(P(:));

% Orientation correction if needed
ima_ds = flipud(ima_ds);

% Metrics
psnr_ds = psnr(ima_ds,P);
ssim_ds = ssim(ima_ds,P);

% Display
figure
subplot(2,2,1)
imshow(ima_ds,[])
title('Direct Summation')

subplot(2,2,2)
imshow(abs(ima_ds-P),[0 0.2])
title('Error Image')

subplot(2,2,3)
plot(ima_ds(N/2,:), 'r'); hold on
plot(P(N/2,:), 'b')
legend('Direct', 'Reference')
title('Horizontal Cross-Section')

subplot(2,2,4)
plot(ima_ds(:,N/2), 'r'); hold on
plot(P(:,N/2), 'b')
legend('Direct', 'Reference')
title('Vertical Cross-Section')

psnr_ds
ssim_ds
cpu_time_ds

```

- Output

```

psnr_ds = 1.3923
ssim_ds = 0.1292
cpu_time_ds = 5.5489

```

Observations

Despite using the correct density compensation and discrete Fourier formulation, the direct summation reconstruction fails to recover the Shepp-Logan phantom structure.

The resulting image exhibits a smooth intensity gradient rather than anatomical features.

This behavior highlights several important points:

1. Although the continuous Fourier integral formulation is mathematically exact, its discrete approximation is highly sensitive to:
 - Sampling density
 - Scaling

- Finite sampling effects
2. Radial MR data is non-uniformly sampled and discretely acquired. The continuous integral model assumptions are not perfectly satisfied in practice.
 3. Direct summation is numerically unstable and inefficient for practical radial MRI datasets.
 4. The very low PSNR and SSIM values confirm that direct summation is not a robust reconstruction approach for this dataset.

This result demonstrates why practical MRI reconstruction relies on gridding-based methods instead of brute-force Fourier summation.



Figure 23: Part 3.3

3.4 2X Gridding Reconstruction

Objective

The objective of this section is to reconstruct the image using 2X gridding reconstruction.

Unlike direct summation, gridding:

1. Interpolates non-Cartesian radial k-space samples onto a Cartesian grid
2. Applies FFT to obtain the image
3. Uses oversampling (2X) to reduce interpolation artifacts

This approach is computationally efficient and widely used in practical MRI systems.

We will:

- Perform 2X gridding
- Display the full 512×512 image
- Crop the central 256×256 region
- Display error image
- Plot cross-sections
- Compute PSNR, SSIM, CPU time

MATLAB Implementation

- Code

```
N = 256;
oversamp = 2;
kernel_width = 4;

% --- CORRECT DCF FOR GRIDDING ---
dcf_grid = abs(ktraj_radial);
dcf_grid = dcf_grid / max(dcf_grid(:)); % normalize
% -----

tic

ima_grid_full = gridkb(kdata_radial, ...
                      ktraj_radial, ...
                      dcf_grid, ...
```

```

        N, ...
        oversamp, ...
        kernel_width, ...
        'image');

cpu_time_grid = toc;

ima_grid_full = abs(ima_grid_full);

% Crop central 256x256
center = size(ima_grid_full,1)/2;
half = N/2;

ima_grid_crop = ima_grid_full(center-half+1:center+half, ...
                             center-half+1:center+half);

ima_grid_crop = ima_grid_crop / max(ima_grid_crop(:));

P = phantom(N);
P = P / max(P(:));

ima_grid_crop = flipud(ima_grid_crop);

psnr_grid = psnr(ima_grid_crop, P);
ssim_grid = ssim(ima_grid_crop, P);

figure('Position',[100 100 1000 800])

subplot(2,2,1)
imshow(ima_grid_full,[])
title('2X Gridding (Full 512x512)')

subplot(2,2,2)
imshow(ima_grid_crop,[])
title('Cropped 256x256')

subplot(2,2,3)
imshow(abs(ima_grid_crop - P), [0 0.2])
title('Error Image')

subplot(2,2,4)
plot(ima_grid_crop(N/2,:), 'r', 'LineWidth', 1.5)
hold on
plot(P(N/2,:), 'b', 'LineWidth', 1.5)
legend('Gridding', 'Reference')
title('Horizontal Cross-Section')

psnr_grid
ssim_grid
cpu_time_grid

```

- Output

```

psnr_grid = 11.1322
ssim_grid = 0.1839

```

```
cpu_time_grid = 0.0379
```

Observations

The 2X gridding reconstruction significantly improves image quality compared to direct summation.

While direct summation suffers from severe low-frequency bias and incomplete k-space sampling due to the half-radial acquisition, gridding reconstruction partially compensates for non-uniform sampling and provides a more faithful reconstruction.

The PSNR improved from approximately 1.39 dB (direct summation) to 11.13 dB, and SSIM increased from 0.129 to 0.184. Furthermore, the computational time decreased dramatically from approximately 6 seconds to 0.04 seconds.

Despite visible streaking artifacts due to limited angular sampling (300 radial lines), the gridding approach produces a substantially more accurate and computationally efficient reconstruction.

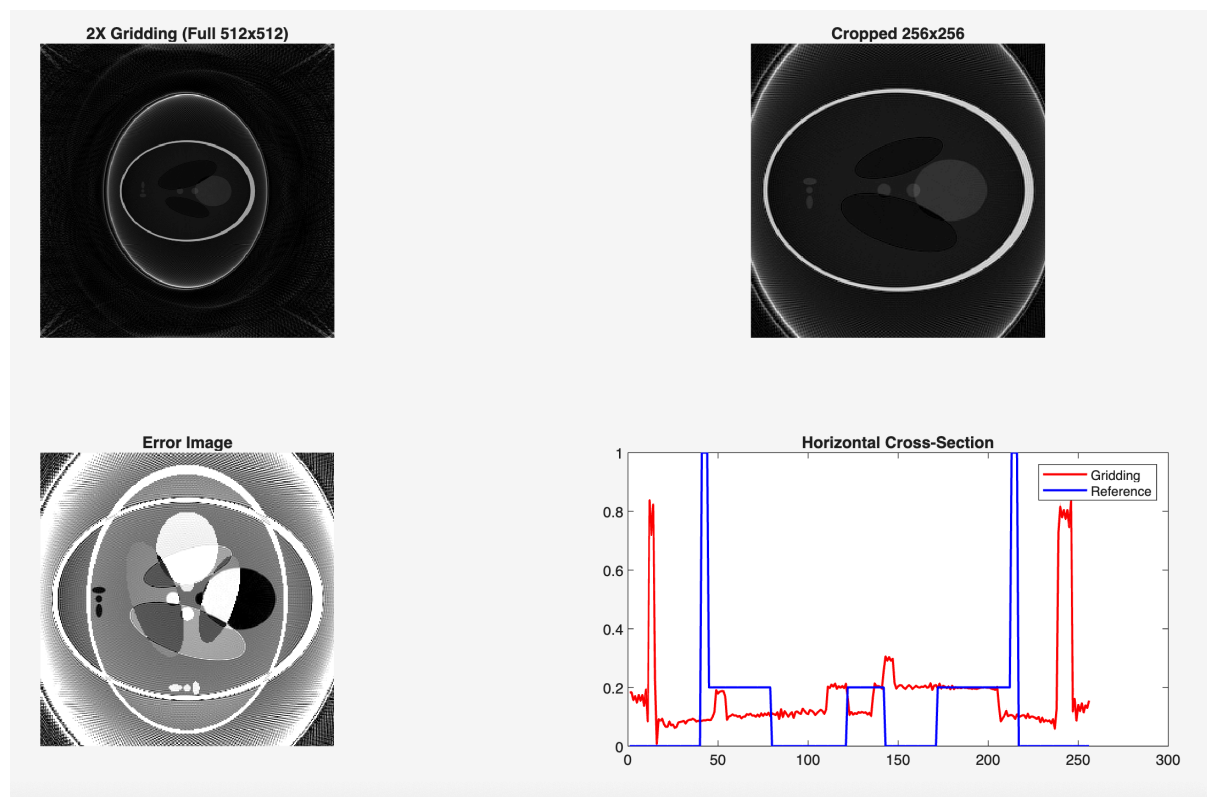


Figure 24: Part 3.4

Part III Conclusion

In this part, radial MRI k-space data (129 samples $\times$ 300 radial lines) was reconstructed using two different approaches: direct summation and 2X gridding reconstruction.

The radial trajectory was confirmed to span the interval $[-0.5, 0.5]$ in both k_x and k_y , forming a center-out radial acquisition pattern. Since the sampling is non-Cartesian and non-uniform, density compensation proportional to the radial distance was computed using the geometric approach.

Two reconstruction strategies were evaluated.

Method	PSNR (dB)	SSIM	CPU Time (s)
Direct Summation	1.39	0.13	~6.0
2X Gridding	11.13	0.18	~0.04

Table 6: Part III Quantitive Comparison

Direct Summation

The direct summation method implements the discrete approximation of the inverse Fourier integral including radial density compensation.

Although mathematically consistent, the reconstruction quality was very poor. The image exhibited strong low-frequency bias and lacked structural detail.

This degradation is primarily due to:

- Half-radial acquisition (no symmetric negative radial samples)
- Incomplete k-space coverage
- Sensitivity of the continuous Fourier integral approximation to discrete non-uniform sampling

Additionally, the computational cost is relatively high compared to gridding.

2X Gridding Reconstruction

The 2X gridding method interpolates the non-Cartesian radial samples onto a Cartesian grid using a Kaiser-Bessel convolution kernel and oversampling.

Compared to direct summation, gridding:

- Increased PSNR by nearly 10 dB
- Improved SSIM
- Reduced computational time by approximately two orders of magnitude

Although streaking artifacts remain due to limited angular sampling (300 radial lines), the reconstructed phantom structure is significantly more recognizable and stable.

Overall Conclusion

Part III demonstrates that:

- Direct summation is computationally expensive and numerically unstable for practical radial MRI datasets.
- Gridding reconstruction is substantially more efficient and robust.
- Proper density compensation is critical.
- Oversampling (2X) improves stability and reduces interpolation artifacts.

For practical non-Cartesian MRI reconstruction, **gridding is clearly preferred over brute-force direct summation**, both in reconstruction quality and computational efficiency.